

What Binarization Costs Virtual Screening Encoders *

Josef Scheifinger

September 2026

Abstract

Virtual screening at a billion-molecule scale is limited less by encoders than by the cost of storing and comparing their output, which is why binary hash codes have become a promising representation. DrugHash proposes learning those codes during training and reports that this is necessary, but it publishes neither code nor weights. This report reproduces two structure-aware virtual screening models, SPRINT and DrugCLIP, on LIT-PCBA and DUD-E and then measures what binarizing their embeddings produces. Both reproductions land within one to two points of their published AUROC, though DrugCLIP’s DUD-E BEDROC comes back 3.2 points low. Against this baseline, two findings were discovered. The two encoders respond to binarization in opposite directions, across 117 SPRINT and 115 DrugCLIP targets: subtracting a shared per-dimension reference before binarizing lifts DrugCLIP above its own baseline by 0.019 AUROC but costs SPRINT 0.28 to 0.30, and even the per-type reference that suits SPRINT best still costs it 0.06 to 0.12. The reason is visible in their per-dimension statistics rather than in the method. Furthermore, post-hoc binarization and a trained hashing objective preserve different halves of the ranking - centering recovers global ranking quality while losing early enrichment and a hash head trained on frozen encoders does the inverse, improving BEDROC by 14% and EF 0.5% by 13.6% while AUROC moves only 2.2%. Because DrugHash’s own $\lambda = 0$ ablation binarized without any centering step, its reported advantage over not hashing is an upper bound on what its training contributes.

1 Story Summary

1. What is the central question?

What does binarizing a virtual screening model’s embeddings actually cost in retrieval quality and does training a hashing objective into the model achieve something that binarizing a finished model cannot?

2. Why is this question important?

Screening libraries contain billions of compounds, so storing one embedding per molecule at 32 bits per dimension runs into terabytes, whereas one bit per dimension would reduce that by a factor of 32. DrugHash reports that a hashing objective has to be trained into the model for this to work, but since neither code nor weights have been published, the claim cannot be easily verified.

3. What evidence/data are needed to answer this question?

A fair reproduction of both models as a baseline, post-hoc binarization measured across several centering schemes on both benchmarks (LIT-PCBA and DUD-E) and several code lengths, and the same objective applied during training so that the two routes are directly comparable.

4. What methods are used to get this evidence/data?

The released checkpoints of SPRINT and DrugCLIP were run on LIT-PCBA and DUD-E with the encoder outputs captured, without the published code being modified. Five centering schemes and five code lengths were then applied to the cached embeddings and a $128 \rightarrow 128$ dimensional L2-normalised head was trained on frozen DrugCLIP encoders under DrugHash’s objective across a λ grid, where λ weights the hashing term in the loss.

5. What analyses must be applied to the data?

AUROC, BEDROC and enrichment factors per target, averaged over targets, together with paired per-target comparisons so that a mean is not driven by a few outliers. Additionally the per-dimension distribution statistics of both encoders were computed to explain why they differ, and the benchmark actives were compared against the list of molecules the released DrugCLIP checkpoint was trained on, in order to measure how much of its benchmark performance could be memorisation rather than retrieval.

6. What evidence/data were obtained?

Both models were reproduced within one or two points on AUROC, and on BEDROC everywhere except DrugCLIP on DUD-E, which is 3.2 points low. The two encoders produced opposite orderings of the centering

*Code and per-target metrics: <https://github.com/josefslink/binary-hashing-virtual-screening>

schemes, identically on both benchmarks, and the trained hashing head improved BEDROC by 14% and EF 0.5% by 13.6% while AUROC increased only 2.2%.

7. What were the results of the analyses?

Subtracting one shared per-dimension reference from both the molecule and the pocket embeddings before binarizing improves global ranking by 0.019 AUROC while lowering BEDROC by 0.027, and the trained hashing objective does the inverse. The difference between the two models is linked to their per-dimension distributions, where DrugCLIP’s mean sits at the 49.9th percentile of its own dimension and SPRINT’s at the 96.7th.

8. How did the analyses answer the central question?

Binarization costs early enrichment rather than global ranking and a trained objective addresses exactly the part that post-hoc binarization loses. Because DrugHash’s own $\lambda = 0$ ablation binarized without any centering step, its measured advantage over not hashing is an upper bound on what the training contributes.

9. What does this answer tell us about the broader field?

A hashing scheme developed on one encoder cannot be assumed to transfer to another, since the effect depends on the geometry of the embedding space rather than on the method alone. Furthermore, compression methods for screening should be judged on early-recognition metrics rather than AUROC.

2 Introduction

Drug discovery is expensive and slow, and virtual screening addresses part of that by ranking molecules computationally before the most promising candidates are tested in a wet-lab. Modern screening libraries are measured in billions of compounds, which means the ranking step has to touch every entry in the library.

2.1 Screening as retrieval

Contrastive dual-encoder methods such as DrugCLIP (Gao et al., 2023) and SPRINT (McNutt et al., 2024) made that step cheap. Both encode protein pockets and molecules into a shared embedding space, so that screening reduces to a similarity search and the expensive encoder runs once per molecule, rather than once per screening task. This is what enables zero-shot screening at a scale traditional docking-based methods cannot reach.

2.2 The memory bottleneck

What remains expensive is the library itself. Storing one billion molecules at SPRINT’s 1024 dimensions in float32 requires roughly 4 TB and even DrugCLIP’s 128 dimensions require roughly 0.5 TB. This is the motivation for binary codes: one bit per dimension instead of 32 is a 32x reduction and Hamming distance reduces the retrieval to an XOR operation followed by a population count, which is computationally very cheap. DrugHash (Han et al., 2024) takes exactly this route, adding a squared-error hashing loss to DrugCLIP’s objective so that the encoder outputs survive being binarized.

2.3 The gap

DrugHash reports that this training is necessary and supports that with an ablation in which λ , the weight on the hashing term, is set to 0. That means no hashing during training but $\text{sign}(\cdot)$ still applied at evaluation. Their conclusion is that accuracy then drops by a large margin. However, that ablation applies $\text{sign}(\cdot)$ directly to the raw embeddings, without first subtracting any reference value, and it is therefore not obvious what a properly configured post-hoc binarization would achieve. Because neither code nor weights were published, there is also no reference implementation against which the claim could be checked.

2.4 Objectives

This report addresses three questions:

1. Do the published LIT-PCBA (Tran-Nguyen et al., 2020) and DUD-E (Mysinger et al., 2012) numbers reproduce from the released checkpoints of DrugCLIP and SPRINT?
2. What does post-hoc binarization of a frozen encoder cost, across centering schemes and code lengths?
3. What does a trained hashing objective add on top of that?

Both models were reproduced on both benchmarks, binarization was applied over cached embeddings and a hash head was trained on frozen encoders under DrugHash’s own objective.

3 Background

3.1 The three methods

DrugCLIP (Gao et al., 2023) treats virtual screening as a cross-modal retrieval problem. Two Uni-Mol (Zhou et al., 2023) encoders, one for protein pockets and one for molecules, are trained contrastively so that a pocket and its true binding partner end up close in a shared 128-dimensional embedding space. At screening time the molecule library is encoded once offline, the query pocket is encoded online and candidates are ranked by cosine similarity.

SPRINT (McNutt et al., 2024) pursues the same retrieval formulation but is built for scale. Its molecule side uses fingerprint features and its protein side uses SaProt (Su et al., 2024), a protein language model whose vocabulary pairs each amino acid with a structural token. Its embeddings are 1024-dimensional, which is eight times DrugCLIP’s and is the reason its memory footprint is the larger of the two.

DrugHash (Han et al., 2024) keeps DrugCLIP’s encoders, training data and contrastive objective entirely unchanged and adds one thing: a squared-error hashing loss that pulls each encoder output towards its own sign pattern, so that the embeddings can be replaced by 128-bit codes and retrieval by Hamming distance. Its novelty is therefore the objective and the retrieval space, not the architecture.

Uni-Core (DP Technology, 2026) is not a model but the framework Uni-Mol and DrugCLIP are built on: DrugCLIP’s task definitions, data pipeline and evaluation entry points are registered through it, so it must be installed for DrugCLIP to run.

3.2 Metrics

Both models rank candidates by the dot product of L2-normalised embeddings and where a target has several pockets the maximum over them is taken.

Three metrics are reported throughout and the distinction between them matters, because the key finding is that they move in different directions. AUROC is the probability that a randomly chosen active is ranked above a randomly chosen inactive, which makes it global: a method can improve it substantially without changing the first 0.5% of the ranking at all. BEDROC (Truchon and Bayly, 2007) weights early ranks exponentially and is bounded in $[0, 1]$. The enrichment factor at 0.5% states how many times more actives appear in the top 0.5% than in a random selection, and is closest to what a screening task requires, since only a very small fraction of candidates is ever tested experimentally.

BEDROC has a parameter α that sets how steeply early ranks are weighted, and the two models are evaluated at different values: SPRINT uses $\alpha = 85.0$, while DrugCLIP’s evaluation code hardcodes $\alpha = 80.5$ even though its paper reports BEDROC_{85} . I kept each model’s own value rather than picking one, because those are the values that reproduce the published tables. The consequence is that BEDROC may be compared between my SPRINT rows and SPRINT’s paper, or between my DrugCLIP rows and DrugCLIP’s paper, but a SPRINT BEDROC should not be compared directly against a DrugCLIP BEDROC.

3.3 DrugHash’s objective

DrugHash’s objective is

$$L = L_c + \lambda \cdot L_{\text{hash}}, \quad (1)$$

where L_c is DrugCLIP’s inherited contrastive loss and L_{hash} penalises the squared distance between each encoder output and its own sign pattern, with the binary code treated as a constant during the encoder update. The published optimal settings are $\lambda = 0.2$ and 128-bit codes (Han et al., 2024).

3.4 Benchmarks

Two benchmarks are used. LIT-PCBA (Tran-Nguyen et al., 2020) contains 15 targets with experimentally confirmed actives and is the harder of the two - both models sit only slightly above chance. DUD-E (Mysinger et al., 2012) contains 102 targets with property-matched decoys, is easier, and is the benchmark on which DrugHash’s published advantage over DrugCLIP is largest.

4 Methods

4.1 Setup, hardware and compute

All experiments were run on a single workstation with a consumer GPU: an NVIDIA RTX 2070 with 8.1 GB of VRAM, an AMD Ryzen 5 3600 with 12 threads and 15 GB of system memory. The pre-trained encoders of SPRINT

and DrugCLIP were never fine-tuned and were only ever run in inference mode, because end-to-end training of either model does not fit into 8 GB. The only component that was trained is the small hash head described in Section 4.5, which has roughly 16.5k parameters per modality and 33k in total, and operates on embeddings that have already been computed, so it trains on cached arrays rather than on the encoders themselves.

Table 1: Approximate runtimes on the hardware above. The dumps write the encoder outputs to disk once; everything in the results section is then computed from those files.

Step	Benchmark	Runtime
SPRINT full pass	LIT-PCBA	~15 min
SPRINT full pass	DUD-E	~30 min (first), ~17 min after
DrugCLIP embedding dump	LIT-PCBA	~45 min
DrugCLIP embedding dump	DUD-E	~30 min
Hash head training	per λ value	2–3 min
Re-scoring one condition	either	~10 s

Including runs repeated after bugs were found, the experiments took an estimated 15 to 20 GPU-hours.

The last row of Table 1 is what made the rest of the report possible on this hardware. Testing six centering schemes on two models and two benchmarks would otherwise mean 24 full passes of 15 to 45 minutes each. Instead the encoder outputs are written to disk once per model and benchmark, and every later experiment only changes how those stored arrays are scored. The encoders are frozen, so the embeddings never change and this gives exactly the numbers a re-run would - at about 10 seconds per condition instead of half an hour.

SPRINT and DrugCLIP have incompatible dependency stacks, so each runs in its own virtual environment.

4.2 Working with the published code without modifying it

The copies of the SPRINT, DrugCLIP and Uni-Core repositories are unmodified and every change this project requires is applied from outside at run time.

DrugCLIP’s evaluation pipeline computes the embeddings, uses them and returns four summary numbers, discarding the embeddings and the labels. That is sufficient for printing its AUROC, but the goal was to use those embeddings. I implemented forward hooks on the two projection heads to capture the embeddings while the upstream loop runs untouched, and the molecule dataset’s collater is wrapped to record the labels in batch order. The resulting arrays are verified rather than assumed correct: on DUD-E the upstream routine does return its own score vector, and recomputing that vector from the captured arrays reproduces it with a maximum deviation of 0.0 on all 100 targets.

4.3 Binarization and centering

Binarizing an embedding means replacing each dimension with a single bit, according to whether its value lies above or below some reference. For a drug matrix D and a target matrix T the code is $B = \mathbb{K}[X - r \geq 0]$ and similarity becomes $1 - d_H/d$, where d_H is the Hamming distance and d the number of bits.

The choice of reference r is the whole question. Five choices were swept, shown in Table 2. Two of them use the mean and two use the median, so both statistics were tested rather than only one.

Table 2: The five centering modes. *Scope* is what the reference is computed over and *statistic* is how it is computed. `per_type` and `global` are the mean-based variants.

Mode	Scope of the reference	Statistic
<code>none</code>	none, threshold at zero	-
<code>per_type</code>	separately per matrix, per dimension	mean
<code>per_type_median</code>	separately per matrix, per dimension	median
<code>global</code>	shared over $[D; T]$, per dimension	mean
<code>global_median</code>	shared over $[D; T]$, per dimension	median

The reason for testing both scopes is that they make opposite assumptions. A *shared* reference subtracts the same vector from molecules and pockets, which keeps them in the same coordinate system - the two encoders were trained to align in that system, so preserving it should help. A *per-type* reference lets each side be centered on its own distribution, which is the better estimate whenever a side has enough rows to compute a reliable statistic from, but it moves the two sides independently and therefore risks breaking the alignment.

The median variants exist because the threshold is ≥ 0 , so a bit is balanced, meaning roughly half the molecules get a 1, only if the reference splits that dimension in half. The mean achieves this only for a symmetric distribution, whereas the median achieves it for any. Whether balance actually helps is answered in Section 5.3.

One case needs further mentioning, because it also produces plausible numbers. Centering a one-row matrix by its own statistic yields exactly zero, so thresholding at ≥ 0 produces an all-ones code, which carries no information and drops that target out of its own evaluation without raising an error. This affects FEN1 and OPRK1 on LIT-PCBA and every DUD-E target, since all of them have a single pocket. Such matrices are left uncentered and a warning is raised, so `per_type` on DUD-E centers only the molecule side. This is a property of the per-type scope rather than a fixable bug - a single row contains no distribution to estimate from - and it is why `global`, whose reference comes from the concatenation of both matrices, is the more meaningful comparison on DUD-E.

4.4 Code length

A d -dimensional embedding yields a d -bit code, so deciding how few bits a model needs is a question about dimensionality. Three reductions were compared:

1. Gaussian random projection followed by a sign, which is the standard locality-sensitive hashing construction for cosine similarity (Charikar, 2002) within the general framework of Gionis et al. (1999), seeded for reproducibility,
2. truncation, keeping the first k dimensions,
3. selection of the k highest-variance dimensions.

The expectation was that random projection should degrade gracefully and that variance ranking should perform best if the signal is concentrated in the high-variance dimensions.

The same projection matrix is applied to both the molecule and the pocket embeddings. This matters because the two are compared to each other. A random projection is a different, random rotation of the space for every seed, so projecting molecules with one matrix and pockets with another would place them in two unrelated coordinate systems, and the resulting similarities would be meaningless. The dangerous part is that this would not crash: both outputs would still have k columns, the Hamming distance would still be computable, and the only symptom would be scores near chance that look like a negative result rather than a bug.

4.5 Trained hash head on frozen encoders

DrugHash trains its encoders end-to-end, which does not fit into 8 GB VRAM. The closest available approximation is to apply the same objective to a small head on frozen encoders and evaluate the result. This was done for DrugCLIP only, since DrugHash itself is built on DrugCLIP and the comparison is therefore like-for-like; no hash head was trained for SPRINT, which is noted as a limitation in Section 6.2. It is also a lower bound on the objective rather than a reproduction of DrugHash, since the encoders cannot adapt to being binarized and that adaptation is most of what DrugHash’s method does.

The head is one $128 \rightarrow 128$ linear layer per modality, with L2-normalised output and identity initialisation. Both of these choices prevent a specific failure, and are worth explaining because without them the experiment would produce a convincing but meaningless result.

Why the output is normalised. L_c compares embeddings by cosine similarity, which depends on the angle between two vectors and not on their length, so rescaling every embedding leaves it unchanged. The hashing loss does depend on length. An unnormalised tanh head can therefore drive L_{hash} towards zero simply by inflating its weights until tanh saturates near -1 and $+1$, which is where the sign pattern already lies, without changing which side of zero any value falls on. Constraining $\|y\| = 1$ removes that route. Measured on cached embeddings, scaling an unnormalised head’s weights up by 160x reduces the hashing loss 40-fold while exactly 0.00% of bits change.

Why the head starts as the identity. At the first optimisation step the head passes its input through unchanged, so binarizing its output must give exactly the same codes, and therefore exactly the same metrics, as binarizing the raw embeddings. That gives a value that is known in advance and can be checked: if an untrained head does not reproduce the post-hoc numbers from Section 5.2, the training pipeline has a bug. It did reproduce them exactly. Training used 66,164 pocket-molecule pairs from DrugCLIP’s training split, covering 16,744 unique pockets. These are entirely separate from the embeddings computed on LIT-PCBA and DUD-E, which are the test data and were never used to fit the head. The epoch was selected on an inner validation split held out by unique pocket rather than by pair, since the same pocket appears in several pairs and splitting by pair would put the same pocket on both sides, which would be data leakage.

The contrastive loss needs a temperature τ , which controls how sharply the softmax separates the correct pair from the others in a batch. DrugCLIP stores this internally as a learned parameter `logit_scale`, where the temperature is its exponential reciprocal. Reading it out of the released checkpoint gives $\exp(\text{logit_scale}) = 13.99$, so $\tau = 1/13.99 = 0.0715$. The DrugCLIP paper defines τ only symbolically and gives no numeric value, so this was read from the checkpoint rather than taken from the text. I used the checkpoint’s own value rather than the published one, so that the head is trained under the same temperature the encoders were trained with.

5 Results

5.1 Reproducibility

Both models reproduce within one to two points of the published AUROC, and of the published BEDROC everywhere except DrugCLIP on DUD-E (0.4733 against 0.5052), as shown in Table 3. The weakest column is SPRINT’s EF 0.5%, which is the statistic most sensitive to small differences in candidate set and preprocessing.

Table 3: Reproduced metrics against the values published in each model’s own paper. The published rows were transcribed from the papers by hand, except DrugCLIP’s LIT-PCBA row, which its evaluation code prints directly alongside its own result and was read from that output. DrugHash is not listed here because it publishes no LIT-PCBA or DUD-E reproduction of DrugCLIP that could serve as a baseline; it is compared separately in Table 5.

Benchmark	Model	AUROC	BEDROC	EF 0.5%	EF 1%	EF 5%
LIT-PCBA	SPRINT, reproduced	0.7253	0.1106	13.71	9.55	5.13
	SPRINT, published	0.7340	0.1230	15.90	10.78	5.29
	DrugCLIP, reproduced	0.5832	0.0598	6.62	5.18	2.18
	DrugCLIP, published	0.5717	0.0623	8.56	5.51	2.27
DUD-E	DrugCLIP, reproduced	0.8071	0.4733	36.46	30.00	10.35
	DrugCLIP, published	0.8093	0.5052	38.07	31.89	10.66
	SPRINT, reproduced	0.6759	0.0700	3.17	3.34	2.77

DrugCLIP is the only model with a published row on both benchmarks, so it is the only one where the two reproductions can be compared against each other. Its DUD-E reproduction sits closer to the published value, at -0.0022 AUROC, than its LIT-PCBA reproduction does, at $+0.0115$. That asymmetry is interpreted in Section 6.

5.2 The centering sweep

The centering sweep produced the same ordering of conditions on both benchmarks for each model and the two orderings are opposite. SPRINT gives `cosine > per_type > none > global`, whereas DrugCLIP gives `global > cosine > none > per_type`. Both orderings hold over these four conditions on two benchmarks, 117 SPRINT and 115 DrugCLIP targets; the median variants do not keep their rank across benchmarks. The full grid is Table 4.

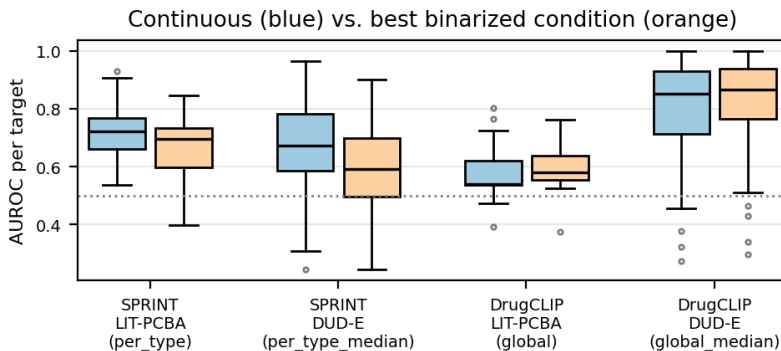


Figure 1: Per-target AUROC: continuous cosine (blue) against the best binarized condition for that model and benchmark (orange, named beneath each group). Boxes show the median and interquartile range over targets, whiskers the range excluding outliers, circles the outliers. The dotted line marks chance. The two models are mirror images of each other: for SPRINT the binarized box sits below its continuous baseline on both benchmarks, whereas for DrugCLIP it sits at or above it. Note also how wide the per-target spread is compared with the differences between conditions.

The result worth emphasising is the *direction*, not the size. Global centering beats DrugCLIP’s own continuous baseline by $+0.0190$ on DUD-E and $+0.0174$ on LIT-PCBA, which on its own is a small gain and is smaller than the spread between the three published DrugCLIP baselines. What makes it notable is that it is positive at all: compressing an embedding from 32 bits per dimension to 1 is expected to cost accuracy, and here it costs nothing. The same operation applied to SPRINT gives 0.4274 and 0.3989, which is worse than random. One preprocessing step therefore moves one model slightly up and the other far below chance, which is a much stronger statement than either number alone.

Figure 1 also shows why the mean values should not be read too confidently. The per-target AUROC spread on DUD-E runs from below 0.3 to 1.0, so a shift of 0.019 in the mean is small against how much individual targets

Table 4: Mean AUROC across all six scoring conditions. Bold marks the best binarized condition per column. Each cell is a re-score of the same cached embeddings, so the conditions differ only in the scoring function.

Condition	SPRINT LIT-PCBA	SPRINT DUD-E	DrugCLIP LIT-PCBA	DrugCLIP DUD-E
cosine (continuous)	0.7253	0.6759	0.5832	0.8071
Hamming, none	0.5442	0.5285	0.5780	0.7918
per_type	0.6662	0.5535	0.5547	0.7885
per_type_median	0.5201	0.5947	0.5407	0.7881
global	0.4274	0.3989	0.6006	0.8261
global_median	0.3533	0.4201	0.5991	0.8280

vary. The reason for trusting it is not its size but that it replicates, with the same sign and magnitude, on a second benchmark.

5.3 Why the two encoders differ

The two orderings are explained by the distributions of the embeddings themselves. For each dimension of an encoder’s output I computed where that dimension’s mean falls within its own values - at the 50th percentile the mean splits the dimension in half, and mean-centering then produces a balanced bit.

DrugCLIP’s distributions are close to symmetric, with the mean at the 49.9th percentile, so mean-centering already produces a balanced code - a bit density (the fraction of bits set to 1) of 0.5008, with only 0.9% of bits differing between mean and median centering. SPRINT’s are extremely heavy-tailed, with a skew of +45.5 and the mean at the 96.7th percentile, so mean-centering produces a sparse code at a bit density of 0.033: a bit is set only for the roughly 3% of molecules with an unusually high value in that dimension. The median forces the density to 0.500 and 46% of bits differ. All of these were computed from the cached embedding dumps, not taken from either paper. Median centering is not a free improvement and for SPRINT on LIT-PCBA it is a large regression, from 0.6662 to 0.5201. For an encoder with SPRINT’s statistics the imbalance is apparently carrying the signal, and forcing a balanced code destroys it.

5.4 Post-hoc binarization against published metrics

Relative to its own continuous baseline, the best post-hoc condition improves global ranking while degrading early enrichment. On DUD-E, global centering gives +0.0190 AUROC, -0.0272 BEDROC and -2.86 EF 0.5%.

Table 5: DrugCLIP on DUD-E, post-hoc binarization against DrugHash’s published result. The **none** row is the like-for-like match to DrugHash’s $\lambda = 0$ ablation, since both binarize without subtracting any reference.

DUD-E	AUROC	BEDROC	EF 0.5%
cosine (continuous)	0.8071	0.4733	36.46
Hamming, none	0.7918	0.4217	32.59
Hamming, global	0.8261	0.4461	33.60
DrugHash, published	0.8373	0.5716	43.03

Against DrugHash’s published row (Han et al., 2024), post-hoc binarization recovers 98.7% of the AUROC but only 78.0% of the BEDROC and 78.1% of the EF 0.5%. The **none** condition, which is the setting DrugHash’s ablation actually tested, loses on every column, at -0.0153 AUROC, -0.0516 BEDROC and -3.87 EF 0.5%, and therefore reproduces their finding.

This is what makes their claim an upper bound rather than a measurement of what hashing contributes. DrugHash measures the value of its training by the gap between $\lambda = 0$ and $\lambda = 0.2$, and attributes that whole gap to the hashing objective. But their $\lambda = 0$ point binarizes without any centering, which Table 5 shows is the worst of the available post-hoc options. A single free preprocessing step recovers 0.0343 AUROC of that gap without any training at all. The part of their reported improvement that is genuinely attributable to training is therefore smaller than the gap they report, and on AUROC specifically it very nearly disappears. Their conclusion does survive for BEDROC and EF, where even the best post-hoc condition remains well short of the trained result.

It should be noted that the $\lambda = 0$ values in DrugHash are plotted rather than tabulated and no AUROC or BEDROC metrics are reported for the different λ values in the ablation. The comparison here therefore rests on their qualitative claim together with the measured numbers in Table 5.

5.5 Trained hash head

Applying DrugHash’s objective to a head on frozen *DrugCLIP* encoders, evaluated on DUD-E, reverses the pattern of the previous section. Figure 2 plots the sweep. Under plain $\text{sign}(\cdot)$, which is what the paper does, increasing λ from 0 to 3 raises AUROC from 0.7853 to 0.8025, BEDROC from 0.4080 to 0.4655 and EF 0.5% from 31.45 to 35.72. Expressed as relative change, this is BEDROC +14%, EF 0.5% +13.6% and AUROC +2.2%, so the gain is concentrated in early recognition and not in global ranking.

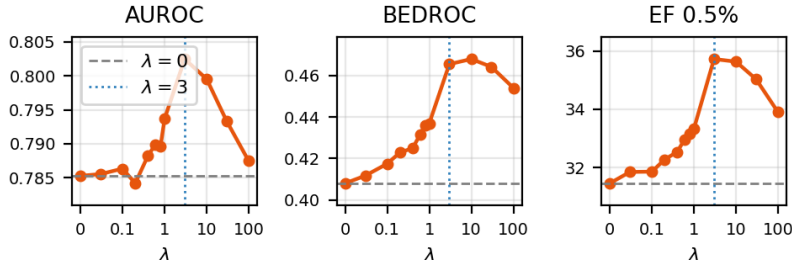


Figure 2: Mean metrics over the 100 DUD-E targets for the trained hash head on frozen DrugCLIP encoders, against the weight λ on the hashing term. The dashed line marks the $\lambda = 0$ baseline and the dotted line the optimum found here. All three metrics improve together, but BEDROC and EF 0.5% rise far more steeply than AUROC. Most of the movement happens beyond $\lambda = 1$, which is where the paper’s grid ends.

A mean over 100 targets can be produced by a few targets improving enormously while most get worse, so the per-target results were also compared pairwise, matching each target’s score at $\lambda = 3$ against its own score at $\lambda = 0$. AUROC improves on 70 of 100 targets, BEDROC on 69 and EF 0.5% on 58, so the effect is broad rather than concentrated in a few outliers.

The trained head also beats the best post-hoc condition from Table 5. Combining $\lambda = 3$ with global centering gives AUROC 0.8333, BEDROC 0.4774 and EF 0.5% 36.34, against 0.8261, 0.4461 and 33.60 for global centering on its own, with BEDROC improving on 65 of 100 targets.

Set against each other the split is stark. Both routes improve AUROC by about two percent - +2.4% for post-hoc centering and +2.2% for the trained head - but centering pays for it with -5.8% BEDROC and -7.9% EF 0.5%, where the trained head gains +14.1% and +13.6% on those same two metrics.

Finally, the λ grid the paper explores turns out to do almost nothing on a frozen encoder. Across the whole range $\lambda \leq 1$, which is where the paper’s grid ends, the validation contrastive loss is flat to three decimal places and the gradient contributed by the hashing term never exceeds 12% of the gradient contributed by the contrastive term - the hashing term is present in the loss but is too weak to change the solution. The optimum found here is $\lambda \approx 3$ on AUROC and EF 0.5%, with BEDROC peaking slightly later at $\lambda = 10$; either way roughly 15 times the paper’s 0.2. This is expected rather than a contradiction: DrugHash can use a small λ because its encoders are also being updated and can move a long way over training, whereas here only a single linear layer can move, so a much larger weight is needed before the hashing term influences the result at all. The practical consequence is that any λ sweep on a frozen encoder has to extend past the published grid, otherwise the flat curve would be misread as evidence that hashing does nothing.

5.6 Code length and a tie-breaking artefact

Truncating SPRINT’s code initially appeared to improve accuracy. LIT-PCBA AUROC rose from 0.6662 to 0.7152 at 64 bits and EF 0.5% from 4.40 to 6.83, monotone in code length and replicating across DUD-E’s 102 targets, which would have been a 16x memory reduction with an accuracy gain.

Table 6 gives the result that survives a fair tie-break; the initial one was entirely an artefact of tie-breaking. Table 6 is the only table in this report scored that way: every other table and both figures use the default actives-first policy, so the hashed numbers in them should be read as upper bounds. A k -bit code admits at most $k + 1$ distinct Hamming similarities, so ties are the norm rather than the exception. The sort used was stable, meaning tied entries keep their original order, and the molecules are loaded actives-first, which means every tie resolved in favour of the actives. Shortening the code multiplies the number of ties, which is exactly why the effect looked cleanly monotone in code length. On MAPK1, one of the 15 LIT-PCBA targets, AUROC falls from 0.7264 to 0.6503 and EF 0.5% from 8.82 to 3.15 once ties are broken at random instead.

With fair tie-breaking the surviving result is less dramatic but more useful. SPRINT tolerates a 16x shorter code at a cost of 0.0146 AUROC on LIT-PCBA and 0.0005 on DUD-E, going from 1024 to 64 bits. Nothing is won by shortening, but almost nothing is lost either. Variance-ranked dimension selection is catastrophic at 0.3883, since the highest-variance dimensions of a heavy-tailed embedding are dominated by a few extreme values and binarize

Table 6: SPRINT under truncation with a seeded random tie-break, so that tied molecules are ordered randomly rather than actives-first.

Bits	LIT-PCBA		DUD-E	
	AUROC	EF 0.5%	AUROC	EF 0.5%
1024 (native)	0.6606	4.15	0.5477	1.89
512	0.6527	3.47	0.5437	1.73
256	0.6483	3.89	0.5423	2.02
128	0.6417	2.81	0.5534	1.96
64	0.6460	2.70	0.5472	2.01

to near-constant bits.

5.7 Training-set leakage

A benchmark active that the model was trained on is being remembered rather than retrieved, so it is worth knowing how many there are. The molecules the released DrugCLIP checkpoint was trained on are published alongside it, so I canonicalised the SMILES strings on both sides with RDKit (RDKit Project, 2026) and counted how many benchmark actives appear in that training set. Table 7 gives the counts.

Table 7: Benchmark actives that also appear in the released DrugCLIP checkpoint’s training ligands.

Benchmark	Actives	Also in training	Targets affected
DUD-E	22,805	583 (2.56%)	90 / 102
LIT-PCBA	9,954	36 (0.36%)	10 / 15

DUD-E’s exposure is roughly seven times higher than LIT-PCBA’s. This applies equally to DrugCLIP’s own published numbers, since it is the same training set, so it is a property of the benchmark and checkpoint pair rather than of this reproduction.

Leakage on the protein side was not checked, and importantly this is not the same as finding none. DUD-E names its pockets by its own target codes, such as **aa2ar**, while DrugCLIP’s training pockets are named by PDB identifier, such as **10gs**. Intersecting the two lists returns zero simply because the two naming systems share no strings at all, not because the pockets are different. Establishing whether there is overlap would require mapping DUD-E targets to PDB entries, which was not available offline.

6 Discussion

6.1 Interpretation

The two encoders differ not in how well they binarize but in what binarization does to them. SPRINT’s heavy tails mean that mean-centering produces a sparse and selective code, in which the imbalance itself carries the signal. DrugCLIP’s near symmetric dimensions mean that a shared origin is precisely what its two encoders were aligned on during contrastive training, which is why removing it hurts the performance and why sharing one helps.

Global centering weakens SPRINT because the shared reference is computed over the concatenation of the drug and target matrices and drugs outnumber targets by four to five orders of magnitude. The shared reference is therefore effectively the drug mean and the target codes end up thresholded against the wrong distribution.

The reproduction asymmetry in Section 5.1 is informative as well. DrugCLIP trains a separate model per benchmark but releases only one checkpoint, whose described training data matches the DUD-E recipe rather than the LIT-PCBA one. The closer DUD-E reproduction is consistent with that and if LIT-PCBA targets were present in the checkpoint’s training data the bias would be upward, which is the direction the +0.0115 delta actually points. This is an inference from the paper and repository rather than something confirmed against the training data.

DrugHash’s own reported gain over DrugCLIP has the same shape as the trained head’s, so two independent routes agree: binarizing a finished encoder improves the global ranking, and training improves the top of the list.

6.2 Limitations

Several features of this setup constrain how the numbers above can be interpreted.

First, the trained head is a lower bound rather than a reproduction. The encoders are frozen and cannot adapt to being binarized and that adaptation is most of what DrugHash does, so the real method should outperform Section 5.4.

Second, 2.56% of DUD-E actives are present in the checkpoint’s training ligands, which means absolute DUD-E values partly measure memorisation rather than retrieval. This applies equally to the published metrics, but it does mean the deltas between conditions are more trustworthy than the absolute values, because they share that exposure.

Third, $\lambda \approx 3$ was identified by looking at the benchmark. The inner-validation objective contains $\lambda \cdot L_{\text{hash}}$ and is therefore not comparable across λ , so it cannot select it. The full curve is reported rather than a single chosen point and the curve is smooth rather than a spike, but the statement that $\lambda \approx 3$ is best remains an observation on DUD-E and not a held-out result.

Fourth, the centering effect is of the same order as the reproduction spread. The +0.0190 gain on DUD-E sits against a 0.0148 spread across the three available DrugCLIP baselines. The reason to believe the effect is that it replicates on LIT-PCBA with the same sign and magnitude, not the DUD-E measurement on its own.

Fifth, hashed enrichment factors are tie-dominated. A 128-bit code admits at most 129 distinct similarity values and only 30 to 76 are observed per target, so the top 0.5% of a library of several hundred thousand molecules is frequently a single tie block. Hashed AUROC should be treated as meaningful and hashed EF as indicative at best. Those ties are also broken in a direction that favours the result: molecules load actives-first and the sort is stable, so every tie in a hashed condition resolves towards an active, while the continuous baseline is effectively tie-free. Section 5.6 shows this was worth 0.0056 AUROC on one SPRINT condition, and the hashed-versus-continuous gaps reported here are not separated from it.

Beyond these, no hash head was trained for SPRINT at all, so the trained-route result rests on a single encoder and the claim that the two routes behave differently is only demonstrated on DrugCLIP. No DTI checkpoint was ever published for SPRINT, so that half of its paper cannot be reproduced, a zero-shot run on BIOSNAP reaches AUPR 0.574 against the 0.858 published for the same 16M model, which is expected rather than a failure. Additionally, nothing here runs at billion scale and Hamming scoring was computed densely rather than through an index, so this work measures accuracy under compression and not retrieval speed.

6.3 Relation to the literature and broader implications

The upper-bound argument of Section 5.4 is the sharpest point this work has to make about the literature, and it is worth being precise about what it overturns. DrugHash’s conclusion really has two parts: that hashing must be trained in to preserve global ranking, and that it must be trained in to preserve early recognition. Centering overturns the first and leaves the second standing.

The transfer implication is the broader one. The effect of binarization depends on the geometry of the embedding space rather than on the hashing method alone, which means a scheme developed on DrugCLIP should not be assumed to work on SPRINT. Since DrugHash is developed on DrugCLIP, this is not a hypothetical concern.

For future work, two consequences follow. A trained hashing objective should be judged on BEDROC and EF rather than on AUROC, since AUROC is already saturated by a free post-hoc step. And SPRINT’s tolerance of a 16x shorter code sets the memory budget such a method would have to work within.

7 Conclusion

Two structure-aware screening models were reproduced within one to two points of their published AUROC, DrugCLIP’s DUD-E BEDROC 3.2 points low, and the cost of binarizing their embeddings was then measured on both.

The two encoders respond to binarization in opposite directions, across 117 SPRINT and 115 DrugCLIP targets on two benchmarks, and the mechanism is visible in their per-dimension statistics rather than in the scoring method. A hashing scheme tuned on one of them should therefore not be assumed to transfer to the other.

Post-hoc binarization and a trained objective preserve different halves of the ranking. Centering a frozen encoder’s codes preserves and slightly improves global ranking quality, whereas a trained objective improves early recognition and two independent routes agree on that observation. The practical consequence is that a hashing objective should be judged on BEDROC and EF rather than on AUROC, which a free post-hoc step already saturates.

The work reported here is an initial exploration rather than a complete evaluation. The trained head runs on frozen encoders and is a lower bound, λ was selected on the benchmark and a measurable fraction of the DUD-E actives were seen during the checkpoint’s training. What the results do establish is that the interesting question is not whether binarization works, but which part of the ranking it preserves and that the answer depends on the encoder as much as on the method.

References

- Moses S. Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of the 34th Annual ACM Symposium on Theory of Computing*, pages 380–388, 2002.
- DP Technology. Uni-Core: An efficient distributed PyTorch framework for deep learning. <https://github.com/dptech-corp/Uni-Core>, 2026. GitHub repository, accessed September 2026; no associated paper.
- Bowen Gao, Bo Qiang, Haichuan Tan, Minsi Ren, Yinjun Jia, Minsi Lu, Jingjing Liu, Weiying Ma, and Yanyan Lan. DrugCLIP: Contrastive protein-molecule representation learning for virtual screening. In *Advances in Neural Information Processing Systems*, 2023.
- Aristides Gionis, Piotr Indyk, and Rajeev Motwani. Similarity search in high dimensions via hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases*, pages 518–529, 1999.
- Jin Han, Yun Hong, and Wu-Jun Li. Hashing based contrastive learning for virtual screening. *arXiv preprint arXiv:2407.19790*, 2024.
- Andrew T. McNutt, Abhinav K. Adduri, Caleb N. Ellington, Monica T. Dayao, Eric P. Xing, Hosein Mohimani, and David R. Koes. Scaling structure aware virtual screening to billions of molecules with SPRINT. *arXiv preprint arXiv:2411.15418*, 2024.
- Michael M. Mysinger, Michael Carchia, John J. Irwin, and Brian K. Shoichet. Directory of useful decoys, enhanced (DUD-E): Better ligands and decoys for better benchmarking. *Journal of Medicinal Chemistry*, 55(14):6582–6594, 2012.
- RDKit Project. RDKit: Open-source cheminformatics. <https://www.rdkit.org>, 2026. Software, accessed September 2026.
- Jin Su, Chenchen Han, Yuyang Zhou, Junjie Shan, Xibin Zhou, and Fajie Yuan. SaProt: Protein language modeling with structure-aware vocabulary. In *International Conference on Learning Representations*, 2024.
- Viet-Khoa Tran-Nguyen, Célien Jacquemard, and Didier Rognan. LIT-PCBA: An unbiased data set for machine learning and virtual screening. *Journal of Chemical Information and Modeling*, 60(9):4263–4273, 2020.
- Jean-François Truchon and Christopher I. Bayly. Evaluating virtual screening methods: Good and bad metrics for the “early recognition” problem. *Journal of Chemical Information and Modeling*, 47(2):488–508, 2007.
- Gengmo Zhou, Zhifeng Gao, Qiankun Ding, Hang Zheng, Hongteng Xu, Zhewei Wei, Linfeng Zhang, and Guolin Ke. Uni-Mol: A universal 3d molecular representation learning framework. In *International Conference on Learning Representations*, 2023.